

LAB 12 : DEQUE

Due Saturday at 5:00 p.m. MST

The eleventh programming assignment of this semester is to write a class with the same functionality as the standard template library (STL) `std::deque` class.

Deque

Your list class will be in `deque.h` and be named `custom::deque`. It will need to support the following operations:

- **Construct:** default constructor and copy constructor.
- **Assign:** copy assign.
- **Iterator:** begin and end.
- **Access:** front, back, and subscript operator. All three are both read-only and read/write.
- **Insert:** push-front (both copy and move) and push-back (also copy and move).
- **Remove:** pop-front, pop-back, and clear.
- **Status:** size and empty.

Iterator

Additionally, we will be creating a deque iterator. Only a read/write forward bidirectional iterator is needed. This will exist in the same `deque.h`. It will need to support the following operations:

- **Construct:** default constructor, copy constructor, non-default construct (so `begin()` and `end()` can properly initialize it).
- **Assign:** copy assign.
- **Compare:** equal-to and not-equal-to.
- **Access:** dereference (read-only and read/write).
- **Arithmetic:** increment (prefix and postfix), decrement (prefix and postfix), difference (for the 100 and 115 options only. When subtracting two iterators, it returns how far apart the two elements are), and add-onto (for the 100 and 115 options. If you add 4 to an iterator, it will skip ahead 4 elements in the deque).

Three Options

This week, there will be three options.

85 Option : List

The 85 option will utilize the **List** implementation as described in the textbook. Unlike previous weeks, this **will** be a template container.

```
template <class T>
```

This will use the standard template library `std::list` container as the underlying data structure. The sole member variable for this option must be called `container`:

```
std::list<T> container;
```

To complete the 85 option, please start with the following files: [232.12.Lab.85.zip](#).

100 Option : Wrapping Array

The 100 option will utilize the **Wrapping Array** implementation. This will utilize a dynamically allocated array as the underlying data structure. This implementation will allow for an arbitrary base type as specified by a template parameter:

```
template <class T>
```

There will be four member variables:

```
T * data;           // dynamically allocated data for the deque
size_t numCapacity; // the size of the data array
size_t numElements; // number of elements in the deque
int iaFront;        // the index of the first item in the array
```

Additionally, the iterator must support the difference operator and the plus-equals operator.

To complete the 100 option, please start with the following files: [232.12.Lab.100.zip](#).

115 Option : Array of Arrays

The 115 option will utilize the **Array of Arrays** implementation and implement a custom allocator. This means there are two template parameters:

```
template <typename T, typename A = std::allocator<T>>
```

There will be no fewer than six member variables:

```
A alloc;           // use allocator for memory allocation
size_t numCells;    // number of cells in a block
size_t numBlocks;   // number of blocks in the data array
size_t numElements; // number of elements in the deque
int iaFront;        // array-centered index of the front of the deque
T ** data;          // array of arrays
```

As with the 100 option, the iterator must support the difference operator and the plus-equals operator. To complete the 115 option, please start with the following files: [232.12.Lab.115.zip](#).

Note that by default, we will use 16 cells per block. However, if the right-hand-side of a copied deque uses a different value for the number of cells, then the copy-constructor will honor that value. In other words, our code should be able to handle any value for `numCells`.

One other detail. There is a bug in the pseudocode for `push_front()` and `push_back()`. We need to call `realloc()` not when the deque is filled, but rather when all the blocks are allocated and the last cell is filled (with `push_back()`, but first cell is filled with `push_front()`). This means the pseudocode needs to be:

```
deque.push back(t)
1. Reallocate the array of blocks as needed
icTail ← (numElements == 0) THEN numCells - 1 ELSE icFromID(numElements - 1)
IF isAllBlocksFilled() AND icTail = numCells - 1
    realloc(IF numBlocks = 0 THEN 0 ELSE numBlocks x 2)

2. Allocate a new block as needed
ib ← ibFromID(numElements)           Find the index of the new block
IF data[ib] = NULL
    dataNew[ib] ← new T[numCells]    Use an allocator or NEW

3. Assign the value into the block
data[ib][icFromID(numElements)] ← t  This is gnarly
numElements++
```

Notice that there is another helper function:

```

/*****
* DEQUE :: IS ALL BLOCKS FILLED?
* return TRUE if all the blocks are filled
*****/
template <typename T, typename A>
bool deque <T, A> ::isAllBlocksFilled() const
{
    // We have no choice but to check each block looking for a NULLPTR
    for (size_t ib = 0; ib < numBlocks; ib++)
        if (nullptr == data[ib])
            return false;
    return true;
}

```

As with last week, the 115 option requires a precise number of allocations and deletions. In order to have more control over this, it is necessary to use an allocator. Please see Appendix I: Allocator in the text book for a detailed description of how this works. There is also a video demonstration here: [Allocators and Spys](#). This explains how the `spy` class works to count allocations, and how `std::allocator` can be use to more precisely control them.

Assignment

Please do the following:

1. Select an option: 85, 100, or 115.
2. Download the code.
3. Implement the data structure with your partner(s).
4. Report how long it took for each individual, and what was the hardest part.
5. Upload your `deque.h` code.

Assessment

The grade for your assignment will be based on the following criteria:

	Exceptional 100%	Good 90%	Acceptable 70%	Developing 50%	Missing 0%
Code Quality 35%	The most elegant solution was found for all the methods; there is no room for improvement,	There are no errors in efficiency, maintainability, cohesion, robustness, or redundancy.	There exists one error in efficiency, maintainability, cohesion, robustness, or redundancy.	The code quality is not what is expected for a CSE 232 student.	The code is sloppy or otherwise unprofessional.
Functionality 50%	Passes all the tests	Passes 90% of unit tests	Passes 70% of unit tests	Passes 50% of unit tests	Program fails to compile or was not submitted.
Wrapping Array 15%	The 100 or the 115 solution was attempted				The 85 solution was attempted
Array of Arrays 15%	115 solution was attempted				The 85 or the 100 solution was attempted